Column subset selection methods in NLA with application to neural networks pruning

Yuekai Yan
Supervisors: Prof. Laura Grigori
Zhipeng Xue

EPFL

yuekai.yan@epfl.ch

June 20, 2026

Overview

1. Background and Motivation
2. Column Subset Selection Problem (CSSP) and Neural Network Pruning
   - Column Subset Selection Problem (CSSP)
   - Neural Network Pruning
3. Methods for Column Subset Selection Problems
   - Strong rank revealing QR (StrongRRQR)
   - Randomly Pivoted Cholesky (RPCholesky)
   - Adaptive randomized pivoting (ARP)
4. CSSP-Based Iterative Pruning
   - Layer Pruning
   - CSSP-Based Iterative Pruning
5. Experiments
   - CSSP-Based Pruning vs. Pruning Filter under FLOPs Constraints
   - Rank Degradation
6. Conclusion and Future Work

Background and Motivation

Background and Motivation

- Nowadays, when pruning a large-scale neural netork, widely used pruning methods are often heuristic and lack explicit approximation guarantees.
- Also, some classical methods in NLA such as SVD approximate activation matrices using linear combinations of all columns, rather than a subset of the original columns, making them less suitable for pruning tasks that require selecting and preserving original features.

Background and Motivation

In this context, methods for the **Column Subset Selection Problem (CSSP)** are especially appealing for neural network pruning:

1. **Low-rank approximation:** CSSP-based methods provide effective approximations of activation matrices, helping preserve activation representations and the prediction behavior of the original model.
2. **Original column preservation:** CSSP preserves the original column structure of activation matrices, making it suitable for pruning tasks and can be combined with other model compression techniques, such as distillation and quantization.

Column Subset Selection Problem (CSSP) and Neural Network Pruning

Column Subset Selection Problem (CSSP)

- The Column Subset Selection Problem (CSSP) seeks a subset of columns with indices $J = (j_1, \ldots, j_r)$ that best approximates the column space of a given matrix.
- More precisely, given a matrix $A \in \mathbb{R}^{m \times n}$, it aims to minimize

$$\|A - \Pi_J A\|_F, \tag{1}$$

where $\Pi_J = A(:, J)A(:, J)^\dagger$.

- (1) cannot be smaller than the best rank-r approximation error:

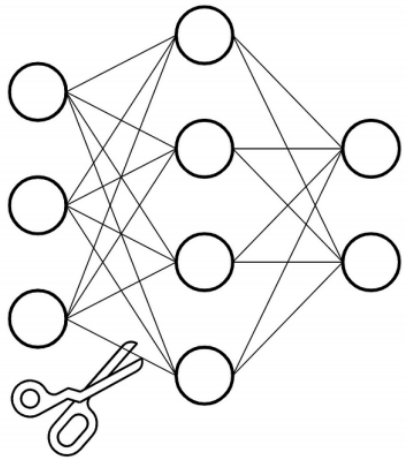
$$\|A - AV_{\text{opt}}V_{\text{opt}}^\top\|_F^2 = \left(\sigma_{r+1}^2(A) + \cdots + \sigma_n^2(A)\right) \leq \|A - \Pi_J A\|_F^2,$$

where matrix $V_{\text{opt}} \in \mathbb{R}^{n \times r}$ represents any orthonormal basis containing right singular vectors belonging to r largest singular values of A.

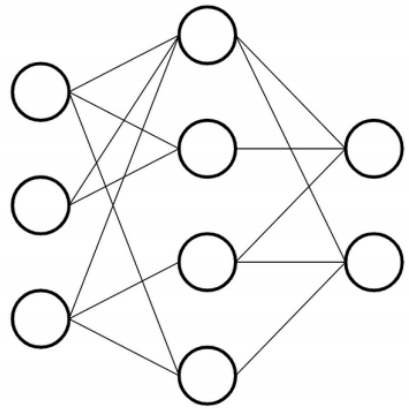
Neural Network Pruning

Neural network pruning aims to reduce model size and computational cost while maintaining predictive performance. Pruning methods can be broadly classified into:

- **Structured Pruning:** removes entire neurons or channels while preserving network structure (e.g., CSSP-based pruning, ℓ_1/ℓ_2 filter pruning).
- **Unstructured Pruning:** removes individual weights while keeping the architecture unchanged (e.g., magnitude pruning).



Before pruning



After pruning

Methods for Column Subset Selection Problems

Strong rank revealing QR (StrongRRQR)

Strong rank revealing QR (StrongRRQR) can be viewed as an enhanced version of QRCP, with additional guarantees on the quality of the selected columns and the conditioning of the resulting factors. It aims to compute, for a matrix $A \in \mathbb{R}^{m \times n}$, a decomposition of the form:

$$AP = Q \begin{pmatrix} A_r & B_r \\ & C_r \end{pmatrix},$$

where $Q \in \mathbb{R}^{m \times m}$ is orthogonal, $A_r \in \mathbb{R}^{r \times r}$ is upper triangular with nonnegative diagonal elements, $B_r \in \mathbb{R}^{r \times (n-r)}$, $C_r \in \mathbb{R}^{(m-r) \times (n-r)}$, and P is a permutation matrix chosen to reveal linear dependence among the columns of A, and the factorization satisfies

$$\sigma_i(A_r) \geq \frac{\sigma_i(A)}{\sqrt{1 + f^2 r(n-r)}}, \qquad \sigma_j(C_r) \leq \sigma_{r+j}(A)\sqrt{1 + f^2 r(n-r)},$$

$$\left|(A_r^{-1} B_r)_{i,j}\right| \leq f, \qquad f \geq 1.$$

Randomly Pivoted Cholesky (RPCholesky)

RPCholesky is a randomized method for approximating a PSD matrix $G \in \mathbb{R}^{n \times n}$ by a rank-r Nyström factorization $G \approx FF^*$. In the context of CSSP, by Gram correspndence, it's equivalent to randomly pivoted QR (RPQR).

RPCholesky

Initialize $F \leftarrow 0_{n \times r}$, $d \leftarrow \mathrm{diag}(G)$, and $J \leftarrow ()$
for $k = 1$ to r **do**
  Sample pivot $j_k \sim d / \sum_{j=1}^{n} d(j)$
  $J \leftarrow (J, j_k)$
  $g \leftarrow G(:, j_k)$
  $g \leftarrow g - F(:, 1 : k-1) F(j_k, 1 : k-1)^*$
  $F(:, k) \leftarrow g / \sqrt{g(j_k)}$
  $d \leftarrow d - |F(:, k)|^2$
  $d \leftarrow \max(d, 0)$
end for

return J, F

Adaptive randomized pivoting (ARP)

Adaptive randomized pivoting (ARP) is a randomized strategy for CSSP. It performs pivot selection an orthonormal basis $V \in \mathbb{R}^{n \times r}$ that approximates the dominant row space of A. For example, V can be efficiently computed via randomized sketching of $A^\top$.

ARP

Initialize $V_0 = V$ and $J = ()$
for $k = 1$ to r **do**
  Set $p_j = \|V_{k-1}(j,:)\|_2^2/(r-k+1)$ for $j = 1, \ldots, n$
  Sample index j_k according to probabilities p_j
  $J \leftarrow (J, j_k)$
  Update $V_k \leftarrow V_{k-1}\Big(I - V_{k-1}(j_k,:)^\dagger V_{k-1}(j_k,:)\Big)$
end for

return J

CSSP-Based Iterative Pruning

Layer Pruning

For a linear layer l, pruning is always performed on the activation matrix $Z_l = h_{1:l}(X; W^{(1)}, \ldots, W^{(l)})$ after the nonlinear transformation, i.e., the ReLU operation. Here, X is the input batch, $W^{(i)}$ denotes the weight matrix of a learnable layer i.

Layer Type	Output Activation Shape	Parameter Shape
Linear	(batch size, out neurons)	(out neurons, in neurons)
Conv	(batch size, out channels, out height, out width)	(out channels, in channels, kernel size, kernel size)

1. Using **column subset selection methods**, we select the column index J of Z_l (neurons to be deleted in layer l):

$$\begin{aligned} Z_l(:, J) &= \mathrm{ReLU}(Z_{l-1}(W^{(l)})^\top)(:, J) \\ &= \mathrm{ReLU}(Z_{l-1}(W^{(l)}(J, :))^\top) \\ &= \mathrm{ReLU}(Z_{l-1}(\hat{W}^{(l)})^\top) \end{aligned}$$

   At this step, we can also get the correction matrix T for the next layer:

$$T = Z_l(:, J)^\dagger Z_l.$$

Layer Pruning

2. Update the next-layer weight matrix using $W^{(l+1)}T^\top$ since:

$$\begin{aligned} Z_l(W^{(l+1)})^\top &\approx Z_l(:,J)Z_l(:,J)^\dagger Z_l(W^{(l+1)})^\top \\ &= Z_l(:,J)(W^{(l+1)}T^\top)^\top \\ &= Z_l(:,J)(\hat{W}^{(l+1)})^\top. \end{aligned}$$

3. Error propagated to the next layer's activation matrix:

$$\begin{aligned} & Z_{l+1} - \hat{Z}_{l+1} \\ &= \|\mathrm{ReLU}(Z_l(W^{(l+1)})^\top) - \mathrm{ReLU}(Z_l(:,J)Z_l(:,J)^\dagger Z_l(W^{(l+1)})^\top)\|_F \\ &\leq \|Z_l(W^{(l+1)})^\top - Z_l(:,J)Z_l(:,J)^\dagger Z_l(W^{(l+1)})^\top\|_F \\ &\leq \|Z_l - Z_l(:,J)Z_l(:,J)^\dagger Z_l\|_F \, \|(W^{(l+1)})^\top\|_F. \end{aligned}$$

CSSP-Based Iterative Pruning

Algorithm (CSSP-Based Iterative Pruning)

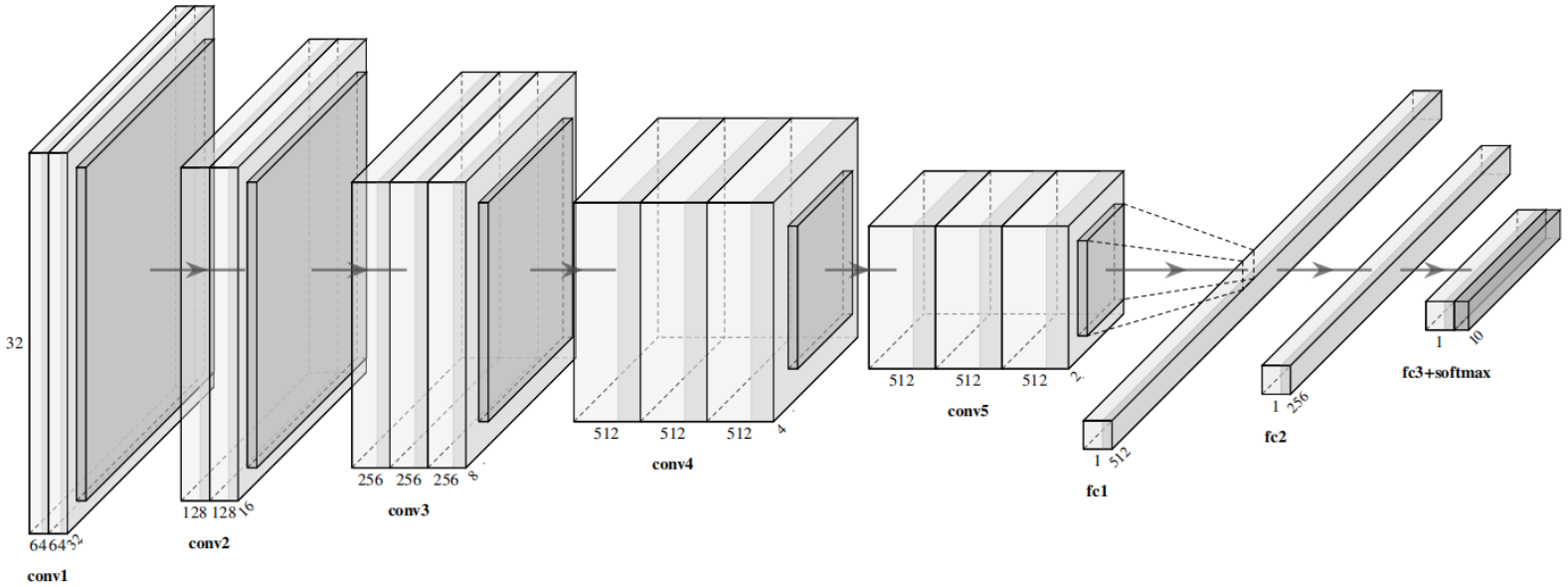
$F_0 \leftarrow$ Compute_Recourse$(h(x; W^{(1)}, \ldots, W^{(L)}))$;
while $F > \rho F_0$ **do**
  scores $\leftarrow \{\}$;
  for $l \in \{1, \ldots, L\} \setminus S$ **do**
    $Z_l \leftarrow h_{1:l}(X; W^{(1)}, \ldots, W^{(l)})$;
    $R \leftarrow$ Pivot_QR(Z_l) {reshape if conv layer}
    $k \leftarrow$ num_channel$(W^{(l)}) \times \lambda$;
    $Err_l \leftarrow \left| \frac{R[k+1, k+1]}{R[1,1]} \right|$;
    $F_l \leftarrow$ Compute_Resource$(\hat{W}^{(l)}, \hat{W}^{(l+1)})$;
    scores. append(Err_l / F_l) ;
  end for
  $l \leftarrow \arg\min(S)$;
  CSSP-based_layer_pruning (layer l) ;
  $F \leftarrow$ Compute_Resource $\left(h(x; W^{(1)}, \ldots, W^{(L)})\right)$;
end while

Prunability scores

Experiments

CSSP-Based Pruning vs. Filter Pruning

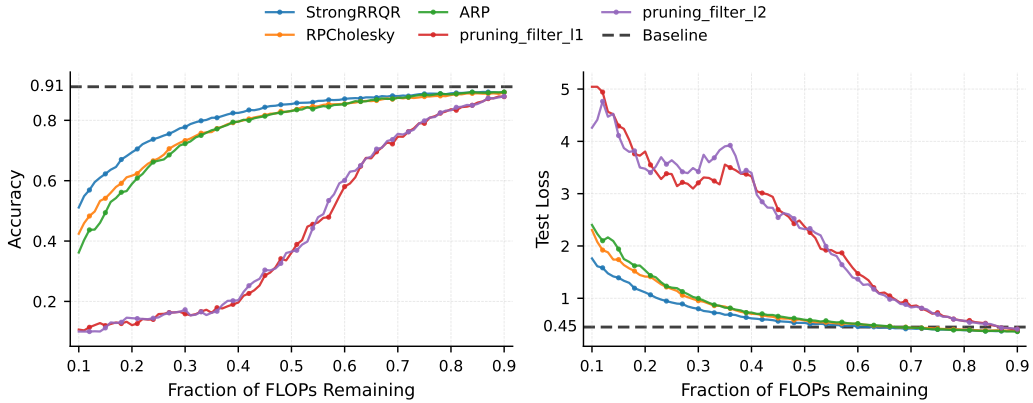
NN Architecture: VGG-16 (5 conv blocks + 3 linear blocks), Dataset: CIFAR-10
10-class image classification task



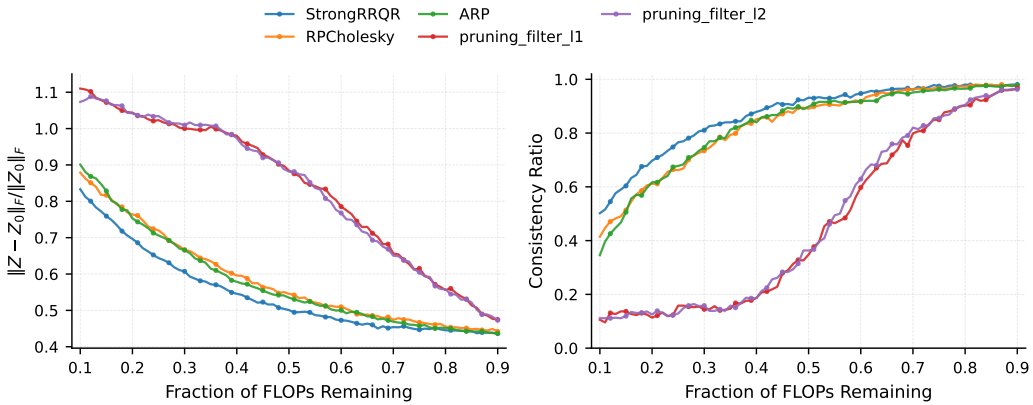
- The pretrained network (unpruned) achieves a classification accuracy of 91.13% with a loss of 0.4497 on test set
- CSSP-based pruning: Strong RRQR, RPCholesky, ARP
- Classical pruning technique: ℓ_1/ℓ_2 filter pruning – remove neurons/channels with small ℓ_1/ℓ_2 norms

CSSP-Based Pruning vs. Filter Pruning under FLOPs Constraints

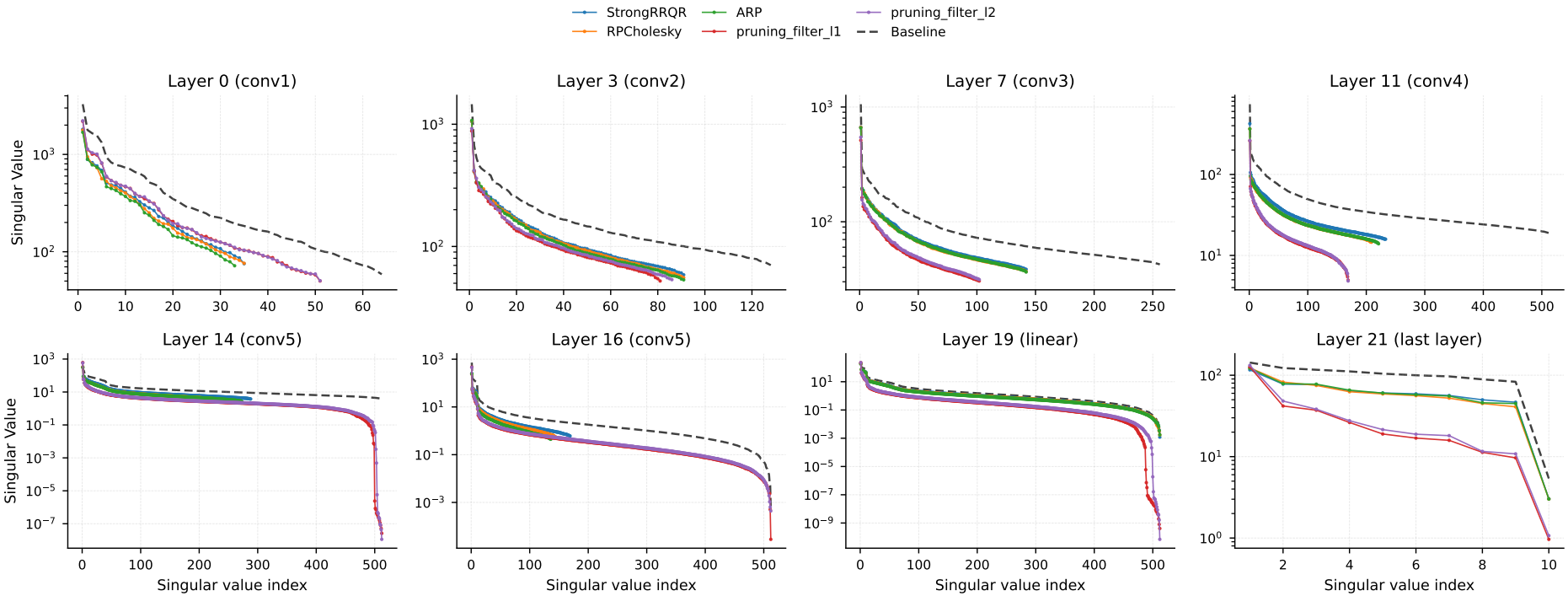
Test accuracy and loss across FLOPs ratios. Dashed line: uncompressed baseline.



Relative Frobenius error $\|Z - Z_0\|_F / \|Z_0\|_F$ & consistency ratio (the proportion of rows whose argmax indices are preserved between Z and Z_0) of the last-layer with size (batch_size, 10)



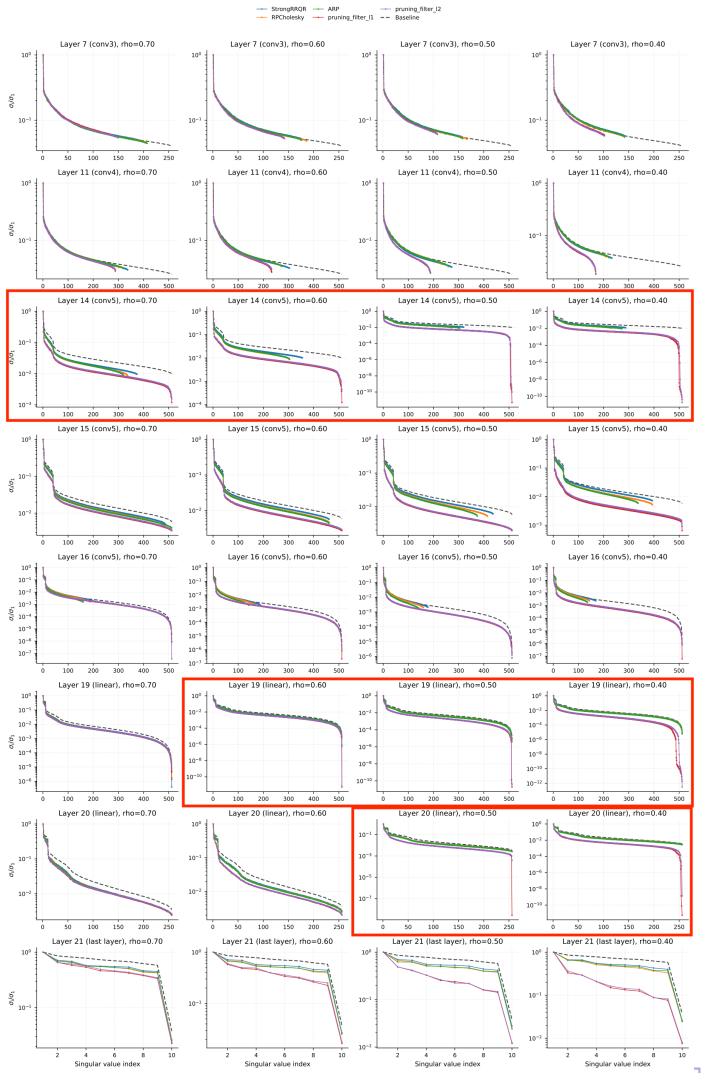
Rank Degradation under 40% FLOPs Retention



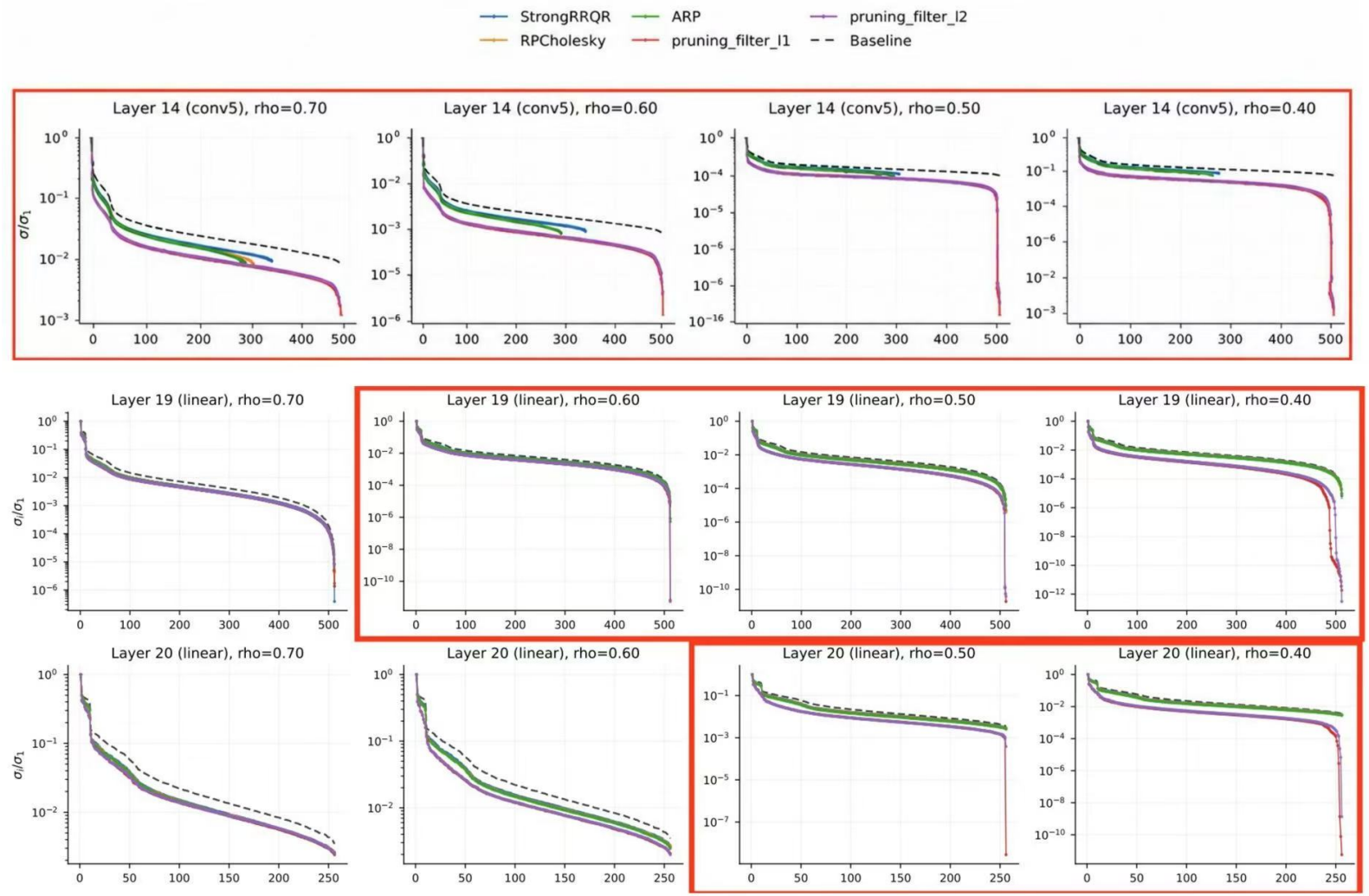
- Rank degradation in deep convolutional and linear layers leads to the poor performance of filter pruning.

Rank Degradation in Filter Pruning

Evolution of relative singular value spectra across layers when the retention ratio decreases from 70% to 40%.

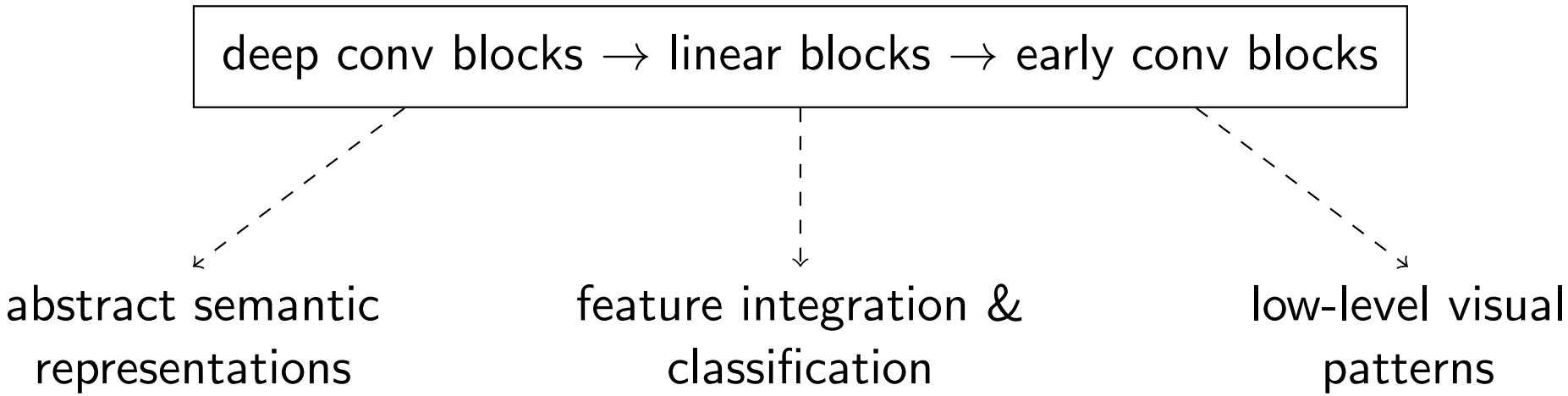


Rank Degradation in Filter Pruning



Rank Degradation in Filter Pruning

- Such a phenomenon is also observed in the experiment on magnitude pruning under parameter constraints. The degradation first appears in some deep layers, and then propagates to the linear layer and the early convolutional layers. [Link]
- The rank degradation of **filter pruning** and **magnitude pruning** during the pruning process appears to exhibit a certain ordering:



Conclusion and Future Work

Conclusion and Future Work

Conclusion

- Preserving the Frobenius norm of the activation matrices is important to preserve the pruned model's performance.
- Different network layers exhibit significantly different pruning sensitivities.
- Compared with classical pruning methods, CSSP-based approaches better preserve activation representations in deeper layers.

Future Work

- Adaptive-rank CSSP with explicit error control.
- Hybrid pruning strategies combining filter pruning and CSSP.

Thank you